Language-feature IRs

Status: decision guide. This guide is scoped to the F# Prelude port (`components/prelude`), not to all F# or Nu. Nu-only features are deferred and must not enter the profile without a concrete Prelude use. It answers one question for every Prelude feature before it is implemented: does it need a dedicated IR like Pattern IR, or is an expression node, a runtime data shape, or a host descriptor enough? It is a gate, not a one-time inventory.

The gate

Answer these in order:

1. Does the feature have its own grammar or a binding sublanguage? If yes, give it an AST.
2. Does it have analyses or invariants that ordinary expressions do not share (binding modes, exhaustiveness, coverage, dispatch, signatures)? If yes, give it an IR and a refusal type.
3. Is it portable, versioned, or sent across an actor/instance boundary? If yes, give it a document schema.
4. Do several consumers need the same structure (parser, evaluator, lowering, verifier, SMT, writer, diagnostics)? If yes, a separate module is justified.
5. Is it only a surface rewrite into existing calls/closures? If yes, keep a node in Expression IR and desugar; do not create a new IR.
6. Is it runtime data? Use an image tag/schema and words, not a language IR.
7. Is it a host effect? Use a capability/effect descriptor and an adapter, not a language IR.

A "yes" at the first four questions is the Pattern IR case. A "yes" only at the fifth is the pipeline/partial case.

Existing-feature substitution review

Before adding an F# feature, check whether an existing Scale mechanism already provides the behavior. The answer is usually "partly": a behavior mechanism is not a data mechanism, and an open mechanism is not a closed one.

F# need	Existing substitute	What it does not replace	Decision
Discriminated unions	Tagged tensors/cells, Pattern IR, multiple dispatch	Closed case set, payload fields, constructors, exhaustive elimination, structural equality. Dispatch selects behavior; it does not define data or destructure it.	Do not substitute dispatch for DUs. Use one tagged-variant model (sum/product) and dispatch for behavior.
Records	Cell classes/slots, association vectors	Nominal vs structural identity, copy-with, structural equality, field patterns. Cell classes are open and mutable by default.	Use a variant/record declaration for immutable data; keep cell classes for identity and mutable instances.
Options and results	Tagged variants, conditions/restarts, host `Result`	The closed two-case shape and total pattern elimination; host `Result` is not a Scale value.	Encode as tagged variants; map exceptions at the host boundary.

F# need	Existing substitute	What it does not replace	Decision
Lists, arrays, tuples	Tensors, vectors, linked cells, value sets	Heterogeneous recursive lists, persistence/sharing, head/tail patterns, tuple arity.	Arrays/tuples from tensors/vectors; lists need a linked-cell or cons model with patterns.
Functions, currying	Closures, code values, explicit _ partials, pipeline stages	General application outside pipelines, automatic currying, named parameters.	No new IR; extend application/lowering and the P4 signature table.
Pattern matching	Pattern IR, guards, decision tables	Data declarations and collection patterns still need the value model.	Keep Pattern IR; do not replace it with dispatch or guards alone.
Modules, namespaces	Package/module document, naming conventions	F# module resolution, opens, access control, compilation units.	Use a module/package document; no expression IR change.
Exceptions	Conditions/restarts, `Result`, named refusals	Stack unwinding, stack traces, `finally` semantics, exception type hierarchy.	Use conditions/`Result` for Scale control; map host exceptions at the adapter.
Classes, interfaces	Cell system, generic functions, multiple dispatch	.NET interfaces, inheritance contracts, member dispatch, object identity interop.	Use cells for language-level identity; host interop stays at the adapter.
Async/tasks	Actors, scheduler, channels	`Task`/`async` await, cancellation tokens, thread-pool semantics, .NET scheduling.	Use actors for Scale concurrency; host tasks are capabilities.
Types, generics, annotations	Dispatch, value sets, arity, signatures	Static inference, parametricity, constraints, overload resolution, type-directed code.	Add a signature/type-declaration layer where the port needs it, not a full F# type checker.
Map/Set	Value sets, association vectors	Ordered maps, hashing, persistent update, the F# library API.	Build library words on image data; do not add a second collection IR.
Macros, computation expressions	None	Compile-time expansion, custom syntax, staging.	A real gap; P6/macros must earn its own design.
BCL/.NET	Host adapters, C ABI, capabilities	Anything not explicitly exposed by the host.	Not a language feature; keep it at the capability boundary.

The key distinction is:

1. A **DU** is closed data with payloads and exhaustive elimination.
2. **Multiple dispatch** is open behavior selected by runtime types/classes.
3. **Cell classes** are open identity-bearing instances.
4. **Value sets** are extensional membership, not closed case declarations.

They compose. None substitutes for another's core invariant.

So: do we need DUs? Not necessarily the exact F# syntax, but we need a closed tagged-variant data model if the ported code pattern-matches cases and carries payloads. Multiple dispatch can implement the operations on those cases, but it cannot replace the cases themselves. If a DU is only an enum-like tag, value sets plus dispatch may be enough; if it is a recursive AST or an error tree, it needs tagged data and pattern extraction.

What other languages do

Feature	Elixir	Julia	Other	Decision here
Functions, application, currying	No automatic currying; `|>` inserts the first argument; functions are values.	No currying; multiple dispatch selects a method from all arguments.	F# curries; Haskell curries; Rust has no currying.	No separate function IR. `ExpressionAst` application/partial/reverse nodes plus closures and a signature table.
Signatures, arity, effects	Typespecs/ Dialyzer, runtime dispatch.	Type annotations and method signatures are semantic dispatch input.	F# inference; Haskell type classes; Rust traits.	Signature/effect table is data, not a pattern-like IR. P4 owns the stack checker.
Records, DUs, options, results	Structs/maps/ tagged tuples; pattern matching is pervasive.	Structs, abstract/ concrete types, `Union`, parametric types.	F# records/DUs; Rust enums; Haskell ADTs.	Yes: a type-declaration IR and portable document, with runtime constructor tags. Pattern matching consumes it.
Pattern matching	Pervasive `=`/`case`/ `with`; guards.	Destructuring and dispatch; no F#-style match.	F# match/active patterns; Rust match; Python 3.10 match.	Yes, already: Pattern IR and its document; keep it separate.
Pipelines	`|>` is a macro that inserts the first argument.	`|>` is function application in dataflow order.	F# `|>` inserts the last argument; `<|` is reverse.	No separate IR: Pipeline nodes preserve surface/style and lower to calls.
Modules, namespaces	Modules are runtime units; `alias`/`import`.	Modules are namespaces; `using`/`import`.	F# modules/ namespaces; Rust crates/ modules.	A module/package document, not a pattern IR. Reuse the package document where possible.

Feature	Elixir	Julia	Other	Decision here
Exceptions, effects	Exceptions via `try/rescue/catch/after`; processes.	Exceptions via `try/catch`; tasks.	F# exceptions; Rust `Result`; Haskell effects.	Conditions/Result at runtime plus host capability descriptors; no pattern-like IR.
Async, tasks, concurrency	BEAM processes and `Task`; message passing.	Tasks, channels, threads; scheduler.	F# async/task; Go goroutines; Rust async.	No language IR: actor/scheduler runtime and host transport; P3/P? actor protocol.
Collections	Lists, maps, structs; immutable.	Arrays, dicts, tuples; mutable arrays.	F# List/Array/Map/Set; Clojure persistent collections.	No IR: runtime data shapes and library words; reuse value sets where structural.
Mutable state, loops	Immutable; recursion/comprehensions.	Mutable structs/arrays; loops.	F# mutable/ref/loops; Rust ownership.	No IR: lower to slots, branches, and the ownership state.

The table is a comparison, not a compatibility promise. Elixir and Julia solve different problems: Elixir leans on macro expansion and process isolation; Julia leans on multiple dispatch and a type/method system. Copying either wholesale would change Scale's semantics.

Forefront decisions

1. **Function values, application, currying.** No new IR module. Use `ExpressionAst.Call`/`Partial`/`Pipeline`/`ReversePipeline`, closures, and the P4 signature table. Currying is nested unary application; the signature table supplies arity and effects. This is the pipeline/partial case, not the Pattern IR case.
2. **Signatures and stack typing.** A signature/effect table is data and a checker, not a pattern IR. It may have a portable document if it crosses a boundary. P4 owns it.
3. **Records, DUs, options, results, tuples.** A type-declaration IR is justified: constructors, fields, generics, equality, and portable shape are shared by the parser, evaluator, pattern compiler, SMT bridge, and writer. Pattern matching then consumes the declarations.
4. **Pattern matching.** Keep Pattern IR it is the canonical case for a dedicated IR because it binds names and has its own analyses.
5. **Modules and namespaces.** Use a module/package document; do not mix module resolution into the expression AST.
6. **Exceptions and effects.** Conditions/Result are runtime values; host effects are capabilities. No dedicated IR unless a new effect algebra earns its own analyses.
7. **Async, tasks, processes.** Actor/scheduler and host transport, not a language IR.
8. **Collections.** Runtime data shapes and words, not an IR.
9. **Mutable state and loops.** Lower to slots/branches; no IR.
10.

+== Rule for every new feature

When a feature is proposed, add one row here and answer the gate explicitly:

1. feature name and motivating Prelude module;
2. grammar/binding sublanguage? (AST or not);
3. analyses/invariants? (IR or not);
4. portable document? (schema or not);

5. consumers? (module or not);
6. runtime data or host effect? (tag/capability);
7. decision, owner phase, and link to the normative document.

No feature is “small enough” to skip the gate: a surface rewrite that should stay a node is as important a decision as a pattern sublanguage that needs its own IR.